Università degli Studi di Udine

Polytechnic Department of Engineering and Architecture

Master's Degree in Management Engineering

Design and implementation of a relational DBMS:

SuperMarket Chain DataBase

Lecturer:

Prof. Francesca Da Ros

Students:

Edoardo Scudeller
Martina Montagner
Lorenzo Gualuppa

Academic Year 2025/2026

Contents

List of Figures

List of Tables

Introduction

In the Large-Scale Retail sector, managing information is one of the most important aspects of a company's success. Every day, a supermarket chain has to manage a huge amount of data coming from very different activities: selling products at the checkouts, controlling goods in the warehouse, managing staff and dealing with suppliers for stock replenishment.

Without an adequate IT system, coordinating all these activities efficiently would be impossible. For this reason, the goal of this project is to design and implement a relational database management system (DBMS) specifically designed to handle the daily operations and business strategies of a generic supermarket chain. The system was developed with the idea of creating a solid, secure and redundancy-free data structure, capable of both supporting routine work in individual stores and providing useful data to management for strategic decisions.

The work was developed step by step following the classic phases of database design. Specifically, the activity was divided into the following phases:

- **Requirements Analysis and Natural Language:** As a first step, all the business rules of the supermarket chain were collected and organized. This phase made it possible to clearly define what information needed to be saved and how the various entities (branches, employees, products, customers...) interacted with each other. The result of this analysis was formalized in the natural language text.
- **Conceptual Design (E-R Schema):** Starting from the natural language text, we moved on building the conceptual model through an Entity-Relationship (E-R) diagram. In this phase, the main entities were identified and the relationships connecting them were established, defining cardinality constraints and the logical assumptions necessary to ensure data consistency.
- **Logical Design (Relational Schema):** The E-R schema was then translated into the logical relational model. During this step, entities became tables and many-to-many relationships were transformed into junction tables. In this phase, all primary keys and foreign keys were precisely defined.
- **SQL Implementation and Query Writing:** Finally, the database was physically created by writing the SQL code (DDL) to generate the tables and their constraints. To demonstrate the system's effectiveness, several SQL queries (DML) of varying complexity were implemented, designed to solve typical problems in supermarket management.

Motivation

The choice to develop a database for a supermarket chain comes from the completeness and richness of this application domain. A supermarket is not just a simple shop, but a complex ecosystem where very different business processes come together every day: supply logistics, human resources management, detailed warehouse control and customer loyalty strategies. Due to its cross-cutting nature and the variety of entities involved, it represents an ideal case study for applying and demonstrating relational design concepts.

In such an articulated environment distributed across multiple local branches, the presence of a Database Management System (DBMS) is not just a computing aid, but an absolutely essential requirement. Managing the continuous flow of thousands of goods, the uninterrupted transit of customers at checkouts and orders to suppliers without a solid database would be completely impossible. In this context, the database becomes the true "heart" of the company: the only tool capable of centralizing information, avoiding data fragmentation, and ensuring that each branch is constantly aligned with the central management.

Therefore, the primary goal of this project is to model and implement an infrastructure capable of governing this complexity, translating it into a consistent and secure data structure. The aim was to create a system that allows managing a global product catalog, while maintaining the flexibility needed to track physical stock and specific prices applied in each local branch.

Furthermore, the system was designed to automate and track daily operations with high precision: from issuing a single receipt at the checkout associated with the respective employee, to automatically updating points on the customers' loyalty cards. Finally, the created infrastructure aims to provide a highly reliable and complete database from which crucial information can be extracted—such as the best-selling products or the top-performing branches—demonstrating how a well-structured database is the cornerstone for ensuring operational efficiency and the commercial success of the entire chain.

1 Requirements and Description in Natural Language

The requirements engineering phase was the fundamental starting point for database modeling. Before writing the natural language description, a careful analysis of the application domain was carried out to extract and classify the company's information needs. This section first presents the identified system requirements and then their translation into the formal natural language text that guided the conceptual design.

1.1 Requirements Analysis

The database requirements were divided into two main categories: data requirements (what information the system must store permanently) and functional/operational requirements (what business logic the system must support). To ensure full coverage of the supermarket chain's operations, the information needs were grouped into the following macro-areas:

- The system requires the management of a centralized product catalog at the corporate level, in order to standardize item coding, descriptions, brands, and standard list prices. At the same time, there is a requirement to map the physical network of stores (branches) in detail. A critical requirement consists in decoupling the concept of a "global product" from that of "local availability": the system must track not only which items are sold by a specific branch, but also the actual stock (quantity) and any price variations applied locally.

- Complete tracking of employee personal data is required. From an organizational point of view, the database must support the internal division of supermarkets into departments. The system requires each worker to be assigned to a specific branch and to the duties of a precise department, thus ensuring a clear hierarchy and the possibility of analyzing staff costs and performance for each operating unit.

- In order to ensure operational continuity, the system needs a module for supplier management. The main requirement is supply chain traceability: it is necessary to store not only the personal and tax data of the supplier companies, but also to map the complex relationship between supplier, product, and branch. The database must uniquely record which specific supplier is responsible for supplying a certain item to a precise geographic branch.

- The system must support the high-frequency recording of purchases (receipts). For each receipt, it is required to save the transaction metadata (date, time, amount, cashier) and the individual detail lines (purchased products, quantity, and actual price at the time of sale). Furthermore, to support marketing strategies, the database must include the management of a loyal customer base (loyalty card holders), allowing transactions to be linked to individual buyers for points accumulation and analysis of consumption behavior.

1.2 Natural Language

Based on the requirements described above, the specifications of the application domain have been formalized in the following natural language text:

We want to design a database for an application concerning a supermarket chain. The database must contain the following information:

- **The stores (or branches),** for each of which an identification code, the city where it is located, the address, and the phone number are known. Each branch has several workers associated with

it. For each branch, there is also interest in the availability for each product, keeping track of the stock quantity and the local price.

- **The global product catalog,** where, for each product in the chain, the following are known: the product code, the category (for example: shampoo, bananas, eggs, etc.), the detail (for example: rigatoni, spaghetti, basmati), the brand, and the standard price. Each product can be sold in multiple supermarkets and each product is associated with a department.
- **The employees,** for whom the following must be stored: the employee code, first name, last name, SSN, role, department, code of the branch where they work, and gender. Each employee works in a supermarket, and the system keeps track of the contract type (for example: fixed-term, permanent, intern, etc.). Employees are divided into two categories: current employees and former employees.
- **The suppliers,** identified by a supplier code, for whom the company name, VAT number, and city of the headquarters are of interest. Each supplier can supply multiple branches in the same area and a supplier can supply multiple products; for this reason, we want to keep track of which supplier provides the single product to the single branch.
- **The customers,** with a loyalty card ID, a first name, a last name, an email, a phone number (optional), and the accumulated points. Each customer is associated with at least one receipt.
- **The receipts,** for which the following are recorded: the receipt number, date, time, total amount, employee code, and loyalty card (also optional). Note that each receipt is associated with exactly one employee.
- **The receipt details,** of which the receipt number (which is referred to), product ID, quantity, and price are of interest. Each detail line refers to a single receipt and a single product.
- **The departments,** with a name and a department code. Multiple employees are associated with each department.

1.3 Assumptions

To resolve any interpretative ambiguities present in the real world, a set of assumptions has been established:

- Each employee is assigned to one and only one department.
- Each product in the catalog must already be present in at least one branch.
- Each product in the catalog has been sold at least once.
- Registered customers have made at least one purchase.

2 E-R Schema

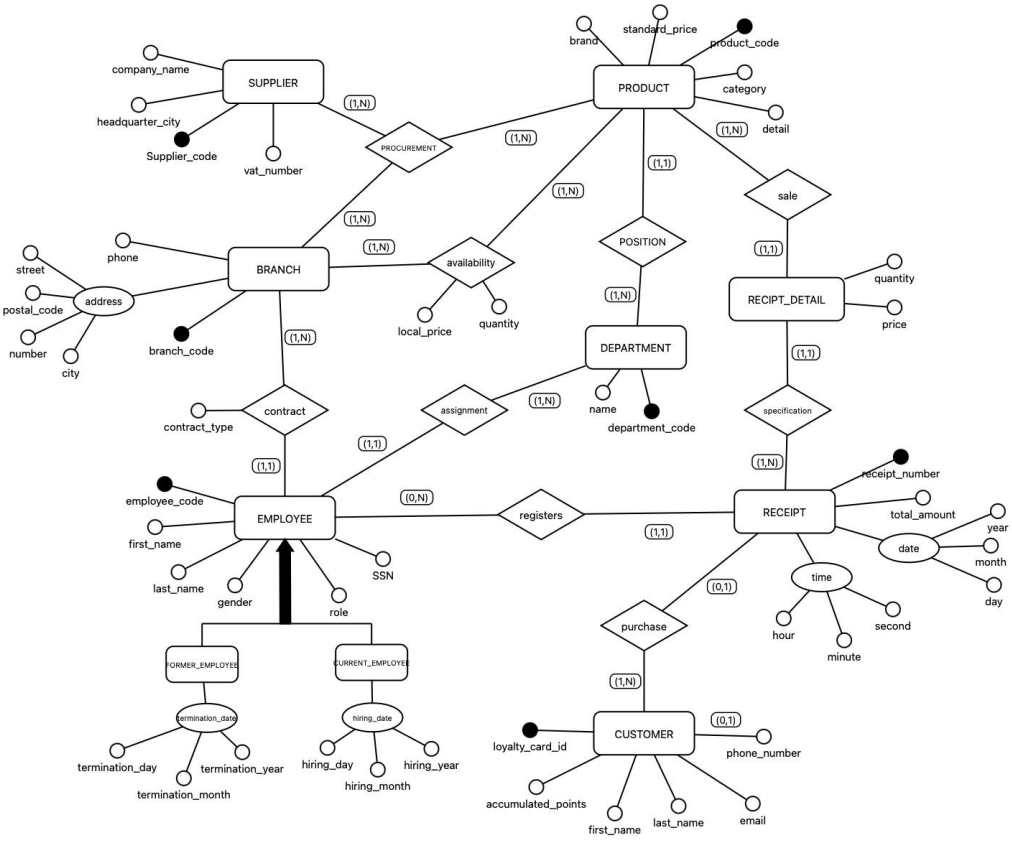

Figure 1: E-R Schema of the SuperMarket Chain database

NOTE: RECEIPT_DETAIL has 2 external identifiers: `product_code`, `receipt_number`.

3 Logical Schema

SUPPLIERS	(supplier_code, vat_number, company_name, headquarter_city)
PROCUREMENTS	(supplier_code, product_code, branch_code)
BRANCHES	(branch_code, phone, street, postal_code, number, city)
EMPLOYEES	(employee_code, SSN, first_name, last_name, role, gender, contract_type, branch, department)
CURRENT_EMPLOYEE	(employee_code, hiring_day, hiring_month, hiring_year)
FORMER_EMPLOYEE	(employee_code, termination_day, termination_month, termination_year)
PRODUCTS	(product_code, brand, category, detail, standard_price, department_code)
AVAILABILITIES	(branch_code, product_code, quantity, local_price)
DEPARTMENTS	(department_code, name)
RECEIPTS	(receipt_number, total_amount, receipt_day, receipt_month, receipt_year, receipt_hour, receipt_minute, receipt_second, employee_code)
CUSTOMERS	(loyalty_card_id, first_name, last_name, email, accumulated_points)
PHONES	(loyalty_card_id, phone_number)
PURCHASES	(loyalty_card_id, receipt_number)
RECEIPT_DETAILS	(product_code, receipt_number, price, quantity)

Table 1: Logical schema

3.1 Foreign Keys

PROCUREMENTS	(supplier_code)	subset of	SUPPLIERS	(supplier_code)
PROCUREMENTS	(product_code)	subset of	PRODUCTS	(product_code)
PROCUREMENTS	(branch_code)	subset of	BRANCHES	(branch_code)
EMPLOYEES	(branch_code)	subset of	BRANCHES	(branch_code)
EMPLOYEES	(department_code)	subset of	DEPARTMENTS	(department_code)
PRODUCTS	(department_code)	subset of	DEPARTMENTS	(department_code)
AVAILABILITIES	(branch_code)	subset of	BRANCHES	(branch_code)
AVAILABILITIES	(product_code)	subset of	PRODUCTS	(product_code)
RECEIPTS	(employee_code)	subset of	EMPLOYEES	(employee_code)
PHONES	(loyalty_card_id)	subset of	CUSTOMERS	(loyalty_card_id)
PURCHASES	(receipt_number)	subset of	RECEIPTS	(receipt_number)
PURCHASES	(loyalty_card_id)	subset of	CUSTOMERS	(loyalty_card_id)
RECEIPT_DETAILS	(receipt_number)	subset of	RECEIPTS	(receipt_number)
RECEIPT_DETAILS	(product_code)	subset of	PRODUCTS	(product_code)
FORMER_EMPLOYEES	(employee_code)	subset of	EMPLOYEES	(employee_code)
CURRENT_EMPLOYEES	(employee_code)	subset of	EMPLOYEES	(employee_code)

Table 2: Foreign key constraints

4 SQL — Sample Data

The following tables present the physical implementation of the database schema in PostgreSQL. For each relation, a sample of 5 randomly selected tuples is shown to illustrate the structure and the nature of the stored data.

4.1 BRANCHES

branch_code	phone	city	street	postal_code	number
23	0131 4960242	Cambridge	Fisher Meadow	B1B 8BQ	826
27	0131 496 0014	Chester	Kimberley Greens	E9 5PS	816
22	0121 496 0935	Derby	Joyce Roads	MK7 0DS	00
17	0151 496 0872	Manchester	Bryan Forge	B5 0YU	820
30	01514960995	Southampton	Williams Spring	W21 8ZP	3

Table 3: BRANCHES — sample tuples

4.2 DEPARTMENTS

department_code	name
BTC	Butchery & Meat
BKE	Bakery
FRZ	Frozen
WNE	Wine & Spirits
CHM	Pharmacy

Table 4: DEPARTMENTS — sample tuples

4.3 SUPPLIERS

supplier_code	vat_number	company_name	headquarter_city
32	GB753876785	Plymouth Frozen Group	Plymouth
18	GB253407200	Brighton Spirits Group	Brighton
28	GB219778234	Belfast Snack Co	Belfast
49	GB330035022	Coventry Magazine Group	Coventry
17	GB328306011	Aberdeen Fashion Co	Aberdeen

Table 5: SUPPLIERS — sample tuples

4.4 PRODUCTS

product_code	category	detail	brand	standard_price	dept_code
323	Oral Care	Fresh Mint Mouthwash 500ml	Listerine	9.92	HBA
372	Bottled Water	Still Spring Water 6x1.5L	Evian	3.41	BEV
693	Magazines	Weekly TV Guide Magazine 1pc	Vogue	4.94	NWS
79	Salad & Herbs	Cherry Tomatoes 250g	Florette	0.90	FRP
151	Shellfish	Raw King Prawns 150g	Whitby Seafoods	9.47	FSH

Table 6: PRODUCTS — sample tuples

4.5 EMPLOYEES

emp_code	first_name	last_name	ssn	role	gender	contract_type	branch	dept
159	George	Holden	RU927065P	Store Manager	M	Permanent	13	CHK
172	Lydia	Johnson	VP952058L	Store Manager	F	Permanent	14	CHK
186	Owen	Bennett	YI258313P	Manager	M	Part-Time	15	CHK
363	Martin	Holloway	MG061497B	Shop Assistant	M	Part-Time	28	WNE
26	Janice	Metcalfe	QT647468F	Shop Assistant	F	Part-Time	2	FRZ

Table 7: EMPLOYEES — sample tuples

4.6 CURRENT_EMPLOYEES

employee_code	hiring_day	hiring_month	hiring_year
201	16	9	2022
105	1	3	1998
190	19	6	2008
142	1	2	2016
149	27	12	1996

Table 8: CURRENT_EMPLOYEES — sample tuples

4.7 FORMER_EMPLOYEES

employee_code	termination_day	termination_month	termination_year
322	5	11	1980
329	17	5	2021
340	24	12	2017
278	14	4	1997
320	30	3	1999

Table 9: FORMER_EMPLOYEES — sample tuples

4.8 CUSTOMERS

loyalty_card_id	first_name	last_name	email	acc_points
2156	Maria	Hussain	maria.hussain122@mail.com	196
928	Andrew	Smith	andrew.smith511@outlook.com	40
1580	Alex	Clarke	alex.clarke221@icloud.com	0
680	Adrian	Robinson	adrian.robinson68@mail.com	66
1562	Eric	Smart	eric.smart798@icloud.com	501

Table 10: CUSTOMERS — sample tuples

4.9 CUSTOMER_PHONES

loyalty_card_id	phone_number
1621	(0113) 4960618
2719	(0117) 496 0940
2530	+44306 9990765
101	0292018684
2658	(0306) 999 0061

Table 11: CUSTOMER_PHONES — sample tuples

4.10 RECEIPTS

receipt_no	day	month	year	hour	minute	second	total	emp_code
7312	13	06	2024	20	15	55	155.69	28
5395	06	05	2024	19	22	36	12.60	236
9489	09	06	2024	16	0	32	123.77	61
5280	16	06	2024	18	39	7	221.39	209
8961	18	02	2024	13	11	35	45.32	212

Table 12: RECEIPTS — sample tuples

4.11 RECEIPT_DETAILS

receipt_number	product_code	quantity	price
4623	709	4	2.15
8116	746	3	6.78
9210	732	2	3.34
7791	275	1	1.34
3407	329	1	9.82

Table 13: RECEIPT_DETAILS — sample tuples

4.12 PURCHASES

loyalty_card_id	receipt_number
229	3205
1191	1425
2227	1100
2516	7488
1543	1422

Table 14: PURCHASES — sample tuples

4.13 AVAILABILITY

branch_code	product_code	quantity	local_price
29	809	37	11.27
9	785	67	5.20
10	255	34	4.42
22	885	13	22.89
17	399	55	3.20

Table 15: AVAILABILITY — sample tuples

4.14 PROCUREMENTS

supplier_code	product_code	branch_code
47	313	7
72	561	24
28	348	16
47	305	4
15	715	29

Table 16: PROCUREMENTS — sample tuples

5 Methodology for Designing SQL Queries

The file 03 Queries.sql contains 19 SQL queries organized in six thematic "scenes". Each scene represents a realistic business scenario of the supermarket chain. The structure is not casual: it starts from a general view of the organization and then goes to more detailed analysis about prices, staff, suppliers, and the product catalog.

5.1 Scene-Based Structure

The queries are grouped in scenes that simulate the needs of different company departments, from general management to HR, from CRM to the purchasing office. This approach makes sure that each query has a clear business purpose: we do not query the database only to show a SQL technique, but to answer a real business question. For example, Query 4 comes from the CRM need to identify customers with more than 500 loyalty points, and Query 13 comes from HR, which wants to find departments with at least 5 employees for training plans.

5.2 Systematic Use of SQL Techniques

We designed the set of queries to systematically use the main SQL techniques studied during the course.

5.3 Reasoning Behind the Structural Choices

Each query is written with a focus on readability and semantic correctness, more than shortness. We always use descriptive aliases (for example `num_products_available`, `branch_city`, `department_name`) so that the result is easy to understand even without knowing the schema in detail.
In some queries we decided to use `LEFT JOIN` instead of `INNER JOIN`. This allows us to keep all the rows from the main table (for example all branches or all customers), even when there is no matching data on the other side. In this way we can also see cases with missing information, such as branches with no products or customers without phone numbers. This is important for reporting, because the absence of data is still useful information for the company.

6 Query Examples

In this section we present five example queries, each one showing a different SQL technique used in the project. For each query, we provide the SQL code, a sample output table, and a short explanation.

6.1 Query 7 – Active and former employees

Scenario: HR wants a complete overview of the personnel history, distinguishing employees who are still active from those who have left the company.

```sql
SELECT e.employee_code, e.first_name, e.last_name, e.role,
       'Active' AS status,
       ce.hiring_year  AS year_event,
       ce.hiring_month AS month_event,
       ce.hiring_day   AS day_event
FROM employees AS e
     JOIN current_employee AS ce
        ON e.employee_code = ce.employee_code
UNION
SELECT e.employee_code, e.first_name, e.last_name, e.role,
       'Ex-employee' AS status,
       fe.termination_year  AS year_event,
       fe.termination_month AS month_event,
       fe.termination_day   AS day_event
FROM employees AS e
     JOIN former_employee AS fe
        ON e.employee_code = fe.employee_code
ORDER BY status DESC, employee_code;
```

Result: This query uses `UNION` to merge two separate result sets into a single list. The first `SELECT` retrieves active employees with their hiring date from the `current_employee` table; the second retrieves former employees with their termination date from `former_employee`. `UNION` automatically removes duplicates and requires both queries to return the same number of columns with compatible types. The literal strings `'Active'` and `'Ex-employee'` act as labels to distinguish the two groups in the final output.

6.2 Query 8 – Products in a branch with price comparison

Scenario: the category manager of branch 1 wants to see which products are available in the store and how the local prices differ from the standard price.

```sql
SELECT p.product_code, category, brand,
       standard_price, branch_code, local_price,
       (local_price - p.standard_price) AS price_difference
FROM products AS p JOIN availability AS a
        ON p.product_code = a.product_code
WHERE branch_code = 1
ORDER BY price_difference DESC, category, p.product_code;
```

Result: This is the slowest query in the project (1345 ms). It uses an `INNER JOIN` between `products` and `availability` to match each product with its local availability record. The calculated column

`(local_price - standard_price)` is evaluated for every row in the `SELECT` clause, producing the price difference on the fly without storing it in the database. A positive value means the local price is higher than the standard; a negative value means it is lower.

6.3 Query 9 – Total revenue per branch

Scenario: the management wants to compare the sales performance of each branch by summing all receipt amounts.

```
SELECT b.branch_code, b.city,
       COUNT(r.receipt_number)       AS num_receipts,
       SUM(r.total_amount)           AS total_revenue,
       ROUND(AVG(r.total_amount), 2) AS avg_receipt_value
FROM branches AS b
     JOIN employees AS e
        ON b.branch_code = e.branch_code
     JOIN receipts AS r
        ON e.employee_code = r.employee_code
GROUP BY b.branch_code, b.city
ORDER BY total_revenue DESC, b.branch_code;
```

Result: This query uses a three-table join to connect branches to their receipts through the employees who issued them, since receipts are linked to employees and employees are linked to branches. Aggregating across multiple joined tables like this is a common pattern in real reporting scenarios. `COUNT(r.receipt_number)` counts the number of transactions, `SUM(r.total_amount)` gives the total revenue, and `ROUND(AVG(...), 2)` provides the average receipt value per branch, all computed in a single query.

6.4 Query 12 – Price statistics by department

Scenario: the purchasing manager wants to understand the price range of products in each department, to evaluate possible repositioning strategies.

```
SELECT p.department_code, name AS department_name,
       COUNT(*) AS number_of_products,
       MIN(standard_price)           AS minimum_price,
       ROUND(AVG(standard_price), 2) AS average_price,
       MAX(standard_price)           AS maximum_price
FROM products AS p
     JOIN departments AS d
        ON p.department_code = d.department_code
GROUP BY p.department_code, name
ORDER BY number_of_products DESC, p.department_code;
```

Result: This query demonstrates the use of multiple aggregate functions together: `COUNT(*)` counts the products in each department, while `MIN`, `AVG` and `MAX` describe the price distribution. `ROUND(..., 2)` is applied to the average to avoid long decimal values in the output. All non-aggregated columns in the `SELECT` clause (`department_code` and `name`) must appear in the `GROUP BY`, otherwise the query would be invalid.

6.5 Query 15 – Best-selling products

Scenario: the purchasing manager wants to identify the best sellers to guide restocking policies and supplier negotiations.

```
SELECT p.product_code, p.category, p.detail, p.brand,
       SUM(rd.quantity)             AS total_quantity_sold,
       SUM(rd.quantity * rd.price)  AS total_revenue
FROM receipt_details AS rd
     JOIN products AS p
        ON rd.product_code = p.product_code
GROUP BY p.product_code, p.category, p.detail, p.brand
ORDER BY total_quantity_sold DESC, p.product_code;
```

Result: This query joins `receipt_details` with `products` to aggregate sales data at the product level. `SUM(rd.quantity)` counts the total units sold across all receipts, while `SUM(rd.quantity * rd.price)` computes the total revenue generated by each product. The expression inside the second `SUM` is evaluated row by row before the aggregation, making it possible to combine two columns in a single aggregate call.

7 Performance Analysis

The execution time analysis shows that all queries run in about one second, with values between 899 ms and 1345 ms. When we executed the queries several times, the execution times were not always the same. They changed a little for each run, because of the database engine and the system load. For this reason, the values shown in the table and in the chart are only approximate. They give a good general idea of the performance, but they are not exact fixed numbers.
This result is acceptable for an educational project with a medium-size dataset, but it is still useful to compare the different queries and understand why some of them are slower than others. The bar chart "Execution Time per Query (in ms)" clearly shows that most queries are close to the average time of about 1038 ms.

- **Fastest query:** Query 4 ("Most loyal customers by points") with an execution time of only 899 ms.
- **Slowest query:** Query 8 ("Products available in a specific branch with price comparison"), which required 1345 ms.

Average time: The average execution time is around **1038 ms** (a little more than 1 second).

From a structural point of view, the slowest query is Query 8, which analyses the products available in a specific branch and compares the local price with the standard price. The query includes a `JOIN` between the products and availability tables and uses a calculated column (`local_price - standard_price`) in the `SELECT` clause.

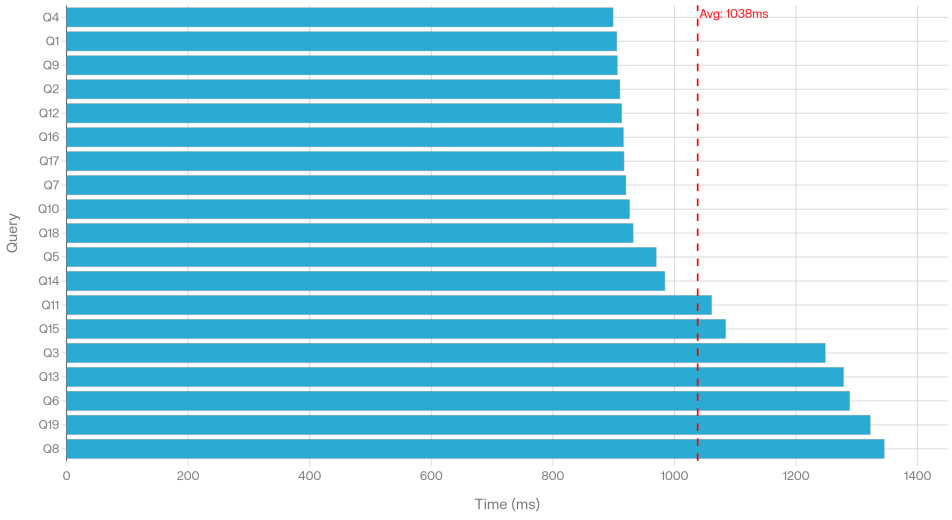

Figure 2: Execution Time per Query (in ms)

7.1 Performance Analysis Details

All times are expressed in milliseconds (ms) to make the comparison easier. The complexity analysis for all queries is classified as "Simple" by the database manager we used.

Query	Description (Scenario)	Time (ms)
Query 4	Most loyal customers by points	899
Query 1	List of branches and products	905
Query 9	Total revenue per branch	906
Query 2	Departments and product distribution	910
Query 12	Statistics by department	913
Query 16	Suppliers without procurements	916
Query 17	Cities as branch and supplier headquarters	917
Query 7	Active vs former employees	920
Query 10	Different roles in the company	926
Query 18	Most expensive products (by category)	932
Query 5	Average spend per loyal customer	970
Query 14	Departments with employees and no products	984
Query 11	Employees list with branch and department	1061
Query 15	Best-selling products	1084
Query 3	Product catalog (overview)	1248
Query 13	Departments with at least 5 employees	1278
Query 6	Customers without phone number	1288
Query 19	Products above the category average	1322
Query 8	Products in branch with price comparison	1345

Table 17: Execution time per query in (ms)

8 Use of Artificial Intelligence as Support

Artificial intelligence tools were used during the development of the queries with a single goal: to generate initial ideas about realistic business scenarios (for example, the type of questions that a CRM manager or a purchasing manager could ask). AI did not replace the design work: the choice of techniques, the scene structure, the aliases, and the ordering criteria are the result of a direct analysis of the schema.

A second area of application involved the generation of realistic synthetic data for populating the database. Given the complexity of the schema, a set of Python scripts was developed across multiple dependency levels: each level handles a specific subset of tables and passes the generated primary keys to the next level through intermediate JSON state files, ensuring referential integrity throughout the process. The scripts make use of dedicated libraries such as Faker (with a British English locale) for generating realistic personal data. In total, the pipeline populated 12 tables with over 165,000 rows, including 3,000 customers, 390 employees distributed across 30 branches, 10,000 receipts with more than 80,000 detail lines, and full availability and procurement records for every product-branch combination.

Finally, AI tools supported the writing and refinement of this explanatory text, especially to organise the ideas in a clear and coherent way. Critical control over the content and the final form of the text was maintained throughout.

Conclusions

This project has demonstrated how a relational database can effectively model and manage the complexity of a supermarket chain, covering all the core operational areas: branch management, product catalog, employee records, supplier tracking, and customer loyalty.

The design process followed the classic phases of database engineering — from requirements analysis to E-R modeling, logical schema translation, and SQL implementation — producing a normalized, redundancy-free structure that supports both daily operations and higher-level business reporting.

The 19 SQL queries, organized in six thematic scenes, confirmed that the schema is not only structurally correct but also practically useful. All queries ran within an acceptable time range (899 ms to 1345 ms), with an average of approximately 1094 ms, which is satisfactory for a medium-size educational dataset.

The five example queries in Section 6 illustrate the variety of SQL techniques applied: from simple filtering (`WHERE` + `ORDER BY`) to aggregation with `HAVING`, correlated subqueries, and set operators such as `EXCEPT`. This systematic coverage shows that the database can answer a wide range of business questions with clean and readable code.

Overall, the SuperMarket Chain DataBase represents a solid and extensible foundation. Future developments could include the addition of indexes to further improve query performance, the introduction of views for common reporting queries, and the implementation of stored procedures to automate recurring operations such as loyalty point updates.